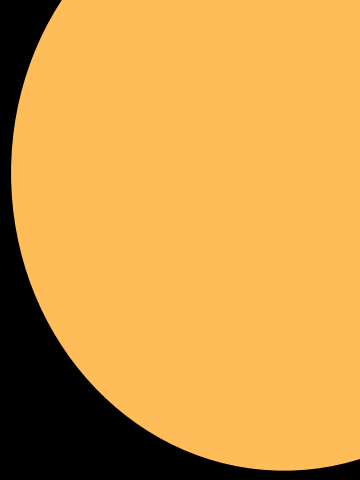




<Coding🎵onata />

# TOP 5 FEATURES IN C# 14

Aram Tchekrekjian

@AramT87

1

# Extension Members

# Extension Members

You can now define **extension properties** and **static extension members** using the new **extension block** syntax.

Extensions can act like **instance members** (called on an object) or like **static members** on the type itself.

You can also define **custom operators** as extension members.

# Extension Members

Add **read-only computed properties** to types you don't control (e.g., `IsEmpty` on `IEnumerable<T>`).

Provide **static helpers** that read like **static members** on the extended type (e.g., `MyType.Identity`).

Build **fluent APIs** without modifying original types.

# Extension Members

```
public static class EnumerableExtensions
{
    // extension block for instance-like members
    extension<T>(IEnumerable<T> source)
    {
        public bool IsEmpty => !source.Any();

        public IEnumerable<T> WhereNot(Func<T, bool> pred)
            => source.Where(x => !pred(x));
    }

    // extension block for static-like members (type receiver)
    extension<T>(IEnumerable<T>)
    {
        public static IEnumerable<T> Identity
            => Enumerable.Empty<T>();

        public static IEnumerable<T> operator
            +(IEnumerable<T> a, IEnumerable<T> b)
            => a.Concat(b);
    }
}

// Usage
var a = new[] {1,2,3};

Console.WriteLine(a.IsEmpty);
// false

var combined = Enumerable<int>.Identity + a;
// uses static extension operator
```

2

# The **field** Keyword



# The field Keyword

With the field keyword you no longer need to declare the **private backing field** when you only need custom logic in an accessor.

The compiler creates a hidden backing variable for you

And then you use the field keyword in the **getter or setter** to read or write that hidden variable.

# The field Keyword

```
public class MessageHolder
{
    public string Message
    {
        get;
        set => field = value ??
                throw new ArgumentNullException(nameof(value));
    }
}

// Usage
var m = new MessageHolder();

m.Message = "hi";
// stores into synthesized backing field

// m.Message = null;
// throws ArgumentNullException
```



3

# First-Class Span Conversions

# First-Class Span Conversions

C# 14 makes `Span<T>` and `ReadOnlySpan<T>` easier to use and more optimized.

You can pass strings, arrays, and spans to methods without extra casting.

Spans let you handle slices of data quickly and safely, giving you high performance without extra memory allocations.

# First-Class Span Conversions

```
using System;

// Method accepting ReadOnlySpan<char>
static void PrintChars(ReadOnlySpan<char> span)
{
    foreach (var c in span)
        Console.Write(c + " ");
    Console.WriteLine();
}

// Method accepting Span<int>
static void DoubleValues(Span<int> span)
{
    for (int i = 0; i < span.Length; i++)
        span[i] *= 2;
}

string text = "Hello";

PrintChars(text);    // implicit conversion in C# 14

// Array automatically converts to Span<int>
int[] numbers = { 1, 2, 3 };

DoubleValues(numbers); // implicit conversion

Console.WriteLine(string.Join(", ", numbers)); // 2, 4, 6
```

4

# Null- Conditional Assignment

# Null-conditional Assignment

Use the null-conditional operators `?.` and `?[]` on the **left side** of an assignment or compound assignment.

The right side is evaluated **only if the left target is non-null.**

With this you can  
Conditionally **write into**  
**nested properties** without a  
**separate null check.**

# Null-conditional Assignment

```
class Customer { public Order? Order { get; set; } }  
class Order { public int Quantity { get; set; } }
```

```
Customer? c = null;  
c?.Order = new Order();  
// safe: no exception, right side not evaluated
```

```
c = new Customer();  
c?.Order.Quantity += 1;  
// compound assignment works when Order != null
```



5

# Lambda Parameter Modifiers

# Lambda Parameter Modifier

Lambda parameters can carry modifiers like `ref`, `out`, `in`, `scoped`, `ref readonly` without you having to write the parameter types.

This makes lambdas `shorter` and `easier to read` when working with delegates that require modifiers.



# Lambda Parameter Modifier

```
delegate bool TryParseInt(string s, out int result);

TryParseInt parser = (s, out result) => int.TryParse(s, out result);

if (parser("123", out var value))
{
    Console.WriteLine(value); // 123
}
```

# Found this useful?



## Consider Reposting

# Thank You

## Follow me for more content



**Aram Tchekrekjian**



**Get Free tips and Tutorials in .NET and C#**



**Join 1,300+ Readers**

**[CodingSonata.com/newsletters](https://CodingSonata.com/newsletters)**

<CodingSonata />